Problem Set 1

Applied Stats II

Due: February 11, 2026

Instructions

- Please show your work! You may lose points by simply writing in the answer. If the problem requires you to execute commands in `R`, please include the code you used to get your answers. Please also include the `.R` file that contains your code. If you are not sure if work needs to be shown for a particular problem, please ask.
- Your homework should be submitted electronically on GitHub in `.pdf` form.
- This problem set is due before 23:59 on Wednesday February 11, 2026. No late assignments will be accepted.

Question 1

The Kolmogorov-Smirnov test uses cumulative distribution statistics test the similarity of the empirical distribution of some observed data and a specified PDF, and serves as a goodness of fit test. The test statistic is created by:

$$D = \max_{i=1:n} \left\{ \frac{i}{n} - F_{(i)}, F_{(i)} - \frac{i-1}{n} \right\}$$

where F is the theoretical cumulative distribution of the distribution being tested and $F_{(i)}$ is the ith ordered value. Intuitively, the statistic takes the largest absolute difference between the two distribution functions across all x values. Large values indicate dissimilarity and the rejection of the hypothesis that the empirical distribution matches the queried theoretical distribution. The p-value is calculated from the Kolmogorov- Smirnoff CDF:

$$p(D \leq d) = \frac{\sqrt{2\pi}}{d} \sum_{k=1}^{\infty} e^{-(2k-1)^2\pi^2/(8d^2)}$$

which generally requires approximation methods (see Marsaglia, Tsang, and Wang 2003). This so-called non-parametric test (this label comes from the fact that the distribution of the test statistic does not depend on the distribution of the data being tested) performs

poorly in small samples, but works well in a simulation environment. Write an `R` function that implements this test where the reference distribution is normal. Using `R` generate 1,000 Cauchy random variables (`rcauchy(1000, location = 0, scale = 1)`) and perform the test (remember, use the same seed, something like `set.seed(123)`, whenever you're generating your own data).

As a hint, you can create the empirical distribution and theoretical CDF using this code:

```
# create empirical distribution of observed data
ECDF <- ecdf(data)
empiricalCDF <- ECDF(data)
# generate test statistic
D <- max(abs(empiricalCDF - pnorm(data)))
```

Question 1 Answer:

- The seed is set to ensure results are replicable
- The data is assigned to 1,000 Cauchy random variables

```
set.seed(123)
test_data <- rcauchy(1000, location = 0, scale = 1)
```

<u>The KS_test function</u>

- An empty vector is initialized and assigned to the variable vector.
- The KS_test function is defined; it takes one argument, data.
- The data argument is turned into a CDF function using the ecdf function and assigned to the variable ECDF.
- The CDF function is used on the data and assigned to the variable empiricalCDF.
- The greatest absolute difference between the empirical CDF and a normal distribution is found and assigned to the variable D.
- A for loop indexes through the data and stores the outcomes of the equation (which is part of the p-value approximation equation for the Kolmogorov-Smirnov CDF) in the vector that was previously initialized.
- The values in the vector are summed together.
- The scaling factor is calculated by taking the square root of 2 * pi and dividing it by D

- The summed vector is multiplied by the scaling factor and assigned to the variable p
- The function returns a vector with D (the test statistic) and p (the approximated p-value)

```
vector <- c()

KS_test <- function(data) {
  ECDF <- ecdf(data)
  empiricalCDF <- ECDF(data)
  D <- max(abs(empiricalCDF - pnorm(data)))
  for(i in 1:length(data)) {
    vector[i] <- exp((-(2 * i - 1)^2 * pi^2)/(8 * D^2))
  }
  sum_vector <- sum(vector)
  scaling_factor <- sqrt(2*pi)/D
  p <- sum_vector * scaling_factor
  return(c(D, p))
}
```

- The KS_test function is used on the test data

```
KS_test(test_data)
```

```
[1] 1.347281e-01 5.652523e-29
```

A test statistic of 1.347281e-01 or 0.1347281 indicates that the largest difference between our empirical cumulative distribution function and a normal distribution function is 0.1347281 on a scale of 0 to 1. The interpretation of the test-statistic is based on the sample size; therefore, it cannot be interpreted without the p-value. However, larger test statistics suggest greater difference between the distribution being tested and the theoretical distribution. Larger test statistics correspond with smaller p-values and vice versa.

The p-value of 5.652523e-29 is very small, well below the commonly used critical value of 0.05. Therefore, we can reject the null hypothesis that the empirical CDF for our data matches the normal distribution.

Question 2

Estimate an OLS regression in `R` that uses the Newton-Raphson algorithm (specifically `BFGS`, which is a quasi-Newton method), and show that you get the equivalent results to using `lm`.

Question 2 Answer

- The seed is set and data is created

```
set.seed (123)
data <- data.frame(x = runif(200, 1, 10))
data$y <- 0 + 2.75*data$x + rnorm(200, 0, 1.5)
```

- The lm function is run, the result of which will be compared to the MLE function.
- I also found sigma squared or the residual variance of the linear model, so I can compare it to the output of the MLE function.

```
lm(y ~ x, data)
summary(lm(y ~ x, data))$sigma^2
```

```
> lm(y ~ x, data)

Call:
lm(formula = y ~ x, data = data)

Coefficients:
(Intercept)            x
0.1392       2.7267

> summary(lm(y ~ x, data))$sigma^2
[1] 2.09322
```

The linear.lik function

- The linear.lik function is defined; it takes 3 arguments, theta, y, and x.
- The variable n is assigned to the number of rows in x
- The variable k is assigned to the number of columns in x
- The variable beta is assigned to the regression coefficients, which are extracted from within theta

- The variable sigma2 is assigned to the squared element in the k+1 position of theta
- The variable e is assigned to y minus the matrix multiplication between x and beta
- The previously calculated variables are used in the log likelihood equation and the calculated result is assigned to the variable logl
- The function returns -logl, it returns the negative log to account for the fact that the optim function finds the minimum of the function and we want the maximum of the function

```
linear.lik <- function(theta, y , x){
  n <- nrow(x)
  k <- ncol(x)
  beta <- theta[1:k]
  sigma2 <- theta[k+1]^2
  e <- y - x%*%beta
  logl <- -.5*n*log(2*pi) -.5*n*log(sigma2) - ((t(e) %*% e)/(2*sigma2))
  return(-logl)
}
```

- The optim function is used with our linear.lik function, the initial parameters all set to one, a hessian matrix is computed, our x and y values are set, and the method is set to BFGS (a quasi-Newton method); the result is assigned to the parameter linear.MLE and the parameters of linear.MLE are printed.

```
linear.MLE <- optim(fn=linear.lik, par=c(1,1,1), hessian=
TRUE, y=data$y, x=cbind(1, data$x), method = "BFGS")

linear.MLE$par
```

```
[1]  0.1398324  2.7265559 -1.4390716
```

- -1.4390716 was squared to find sigma squared or the residual variance.

```
residual_variance <- (-1.4390716)^2
residual_variance
```

```
[1] 2.070927
```

From the linear regression run with the lm function, the intercept was 0.1392, the beta coefficient for x was 2.7267, and sigma squared was 2.09322. From the linear.lik function, the intercept was 0.1398324, the beta coefficient for x was 2.7265559, and the calculated sigma squared was 2.070927. This suggests that we got very close to the values from the lm function. This indicates that Ordinary Least Squares and Maximum Likelihood will have approximately the same results when used to estimate a linear regression (although this may not always be true of the variance in smaller samples). While OLS is still more useful when our assumptions are met (because it gives an unbiased estimate of variance), MLE can be used in cases when the assumptions for linear regression using OLS are not met.